Efficiency of Algorithms
Time Complexity



Thareendhra Keerthi Wijayasiriwardhane

Department of Industrial Management
Faculty of Science
University of Kelaniya
thareen@kln.ac.lk





Learning Outcomes

- On completion of this session, you should be able to
  - Define the time complexity of an algorithm
  - Explain the importance of the analysis of time complexity
  - Identify factors on which the time complexity depends
  - Determine the time complexity using Operation Count and Step Count approaches
  - Analyze the time complexity of a given algorithm

Time Complexity

- Amount of time that a program written with a particular algorithm needs to run to its completion
- Why are we interested?
  - Some problems require a real-time response to their solution
  - Will prefer the program that runs fast when there are several algorithms for a given problem
  - To guarantee the users that the program written with our algorithm can run to its completion without any time-out

Factors of Time Complexity

- Depends on factors that the space complexity depends
- Like space complexity, some factors are sensitive to the problem being solved
  - Example
    A program that sorts an array of n integers
  - Time complexity is a function of n (instance characteristic)

Time Complexity

- Time complexity of algorithm A

  $$T(n) = c + T_A(n)$$

  where c is insensitive (constant) time contribution and T_A is the sensitive time contribution which is a function of instance characteristic n

Determining Time Complexity

- Two approaches
  - Operation Count
  - Step Count

Operation Count Approach

- Identify one or more key operations of the algorithm
- Time complexity is estimated by determining number of times these key operations are performed
- Success depends on the ability to identify the operations that contribute most to the time consumption

Performance Analysis

- Example 1

```
ALGORITHM max(REF a[] INTEGER, n INTEGER)
   locates max integer in array and returns index
1  i ← 0
2  c ← 1
3  LOOP (c < n)
4     IF (a[i] < a[c])
5        i ← c
6     END IF
7     c ← c + 1
8  END LOOP
9  RETURN i
END max
```

Performance Analysis

- Example 2

```
ALGORITHM find(REF a[] INTEGER, n INTEGER, x INTEGER)
   finds x in array and returns index if found, else -1
1  i ← -1
2  c ← 0
3  LOOP (c < n AND x ≠ a[c])
4     c ← c + 1
5  END LOOP
6  IF (c ≠ n)
7     i ← c
8  END IF
9  RETURN i
END find
```

Step Count Approach

- T(n) is estimated by accounting the time spent for all program steps
- Time complexity is estimated by determining the number of steps that program takes to complete

Performance Analysis

- Algorithm 1

```
ALGORITHM sum(REF a[] INTEGER, n INTEGER)
   returns sum of n integers in array
1  s ← 0
2  c ← 0
3  LOOP (c < n)
4     s ← s + a[c]
5     c ← c + 1
6  END LOOP
7  RETURN s
END sum
```

Programming Instrumentation

- Program 1

```
int g = 0; //global variable
int sum(int a[], int n)
{
    int s = 0;
    g++; //for declaration and initialization of s
    int c = 0;
    g++; //for declaration and initialization of c
    while (c < n)
    {
        g++; //for evaluation of c < n expression
        s += a[c];
        g++; //for s += a[c] addition and assignment
        c++;
        g++; //for c++ increment
    }
    g++; //for last evaluation of c < n expression
    g++; //for return s
    return s;
}
```

Performance Analysis

- Algorithm 2

```
ALGORITHM sum(REF a[] INTEGER, n INTEGER)
   returns sum of n integers in array
1  IF (n > 0)
2     RETURN a[n-1] + Sum(a, n-1)
3  END IF
4  RETURN 0
END sum
```

Programming Instrumentation

- Program 2

```
int g = 0; //global variable
int Sum(int a[], int n)
{
   g++; //for if condition evaluation
   if (n > 0)
   {
     g++; //for addition, recursive call and return
     return a[n-1] + sum(a, n-1);
   }
   g++; //for return statement
  return 0;
}
```

Questions